



INSTITUTO POLITÉCNICO
NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Ingeniería en Inteligencia Artificial

Procesamiento Digital de Imágenes

Práctica 8: Filtros Espaciales y Operadores de Convolución

Profesor:

Saul de La O Torres

Alumno:

Eduardo Uriel Velázquez Arrieta

Índice

1. Introducción	2
2. Desarrollo	2
2.1. Arquitectura del Sistema	2
2.2. Implementación de Kernels	3
2.3. Resultados Experimentales	3
3. Conclusión	7

Índice de figuras

1. Imagen 1 filtro negativo.	4
2. Imagen 2 PB Gaussiano 5×5	4
3. Imagen 3 Laplaciano con 8 vecinos.	4
4. Imagen 4 Sobel combinado	5
5. Imagen 5 prewitt suma	5
6. Imagen 6 Gauss 7×7	5
7. Imagen 7 Gauss 9×9	6

1. Introducción

En el campo del Procesamiento Digital de Imágenes, el filtrado espacial es una técnica fundamental que opera directamente sobre los píxeles de una imagen para mejorar su calidad, extraer características o prepararla para etapas posteriores de análisis computacional. Una de las operaciones matemáticas más importantes para llevar a cabo este filtrado es la convolución discreta.

A través de una matriz bidimensional conocida como *kernel* o máscara, la convolución calcula el nuevo valor de un píxel basándose en una combinación lineal de sus vecinos espaciales. Dependiendo de los pesos numéricos asignados dentro del kernel, es posible aplicar distintos tipos de filtros:

- **Filtros Pasa Bajas:** Actúan como promediadores, atenuando las altas frecuencias. Se utilizan comúnmente para la eliminación de ruido y el suavizado o desenfocado de la imagen (ej. Filtro Gaussiano, Promediador).
- **Filtros Pasa Altas:** Realzan los cambios bruscos de intensidad en la imagen, destacando bordes, contornos y detalles finos. Se basan en aproximaciones discretas de derivadas de primer y segundo orden (ej. Sobel, Prewitt, Laplaciano).

El objetivo de la presente práctica es diseñar e implementar un sistema de software robusto capaz de aplicar una amplia gama de operadores espaciales mediante convolución bidimensional.

2. Desarrollo

El sistema fue desarrollado en el lenguaje de programación Java, adoptando el patrón de diseño arquitectónico Modelo-Vista-Controlador (MVC) para asegurar un código modular, mantenible y escalable.

2.1. Arquitectura del Sistema

1. **Modelo (ConvolucionModel):** Contiene toda la lógica matemática e iterativa. Implementa un motor general de convolución que recorre la imagen en escala de grises y aplica el kernel definido. Se diseñaron métodos para calcular la magnitud del gradiente combinando dos kernels (como en el caso de Sobel H_r y H_c) y métodos para operadores de compás de 8 direcciones (OR y Suma direccional). Para evitar el desbordamiento de bits, se implementó una función *clamp* limitando los valores resultantes al rango $[0, 255]$.
2. **Vista (ConvolucionView):** Proporciona la interfaz gráfica de usuario (GUI) utilizando `javax.swing`. Muestra simultáneamente la imagen original, la imagen procesada y los histogramas correspondientes a ambas, lo que permite un análisis comparativo visual inmediato de la distribución de frecuencias de los píxeles.

3. **Controlador (ConvolucionController):** Actúa como intermediario, gestionando los eventos de la interfaz (carga de archivos y selección de filtros) e invocando los algoritmos del modelo para finalmente actualizar la vista.

2.2. Implementación de Kernels

Dentro de la lógica matemática, se programaron diversas familias de operadores. A continuación, se destaca la aproximación del filtro de Sobel, ampliamente utilizado por su resistencia al ruido gracias al mayor peso en su fila/columna central:

```
1 public double [][] sobelX() {  
2     return new double [][] {{-1,0,1},{-2,0,2},{-1,0,1}};  
3 }  
4  
5 public double [][] sobelY() {  
6     return new double [][] {{-1,-2,-1},{0,0,0},{1,2,1}};  
7 }  
8
```

Listing 1: Implementación de máscaras de Sobel en Java

Además de los operadores de gradiente clásicos, se implementó un conjunto exhaustivo de operadores de Compás (Prewitt, Kirsch, Robinson de 3 y 5 niveles), los cuales buscan la máxima respuesta al contorno en 8 orientaciones espaciales distintas. Finalmente, se integraron operadores basados en la segunda derivada, específicamente el Laplaciano de Gaussiano (LoG) con ventanas de 7×7 y 9×9 .

2.3. Resultados Experimentales

Para la validación empírica del sistema de procesamiento, se procedió a cargar imágenes con distintos niveles de textura y contraste (utilizando fotogramas de Evangelion, Perfect Blue y Berserk como datos de prueba) para observar el comportamiento de las diferentes máscaras convolucionales.

Como se puede observar en la Figura 1, el uso de filtros direccionales permite extraer características específicas de los objetos retratados.

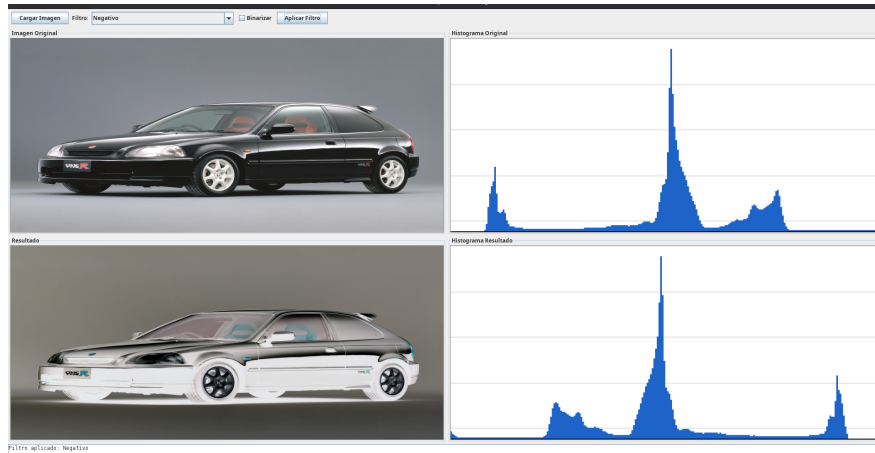


Figura 1: Imagen 1 filtro negativo.

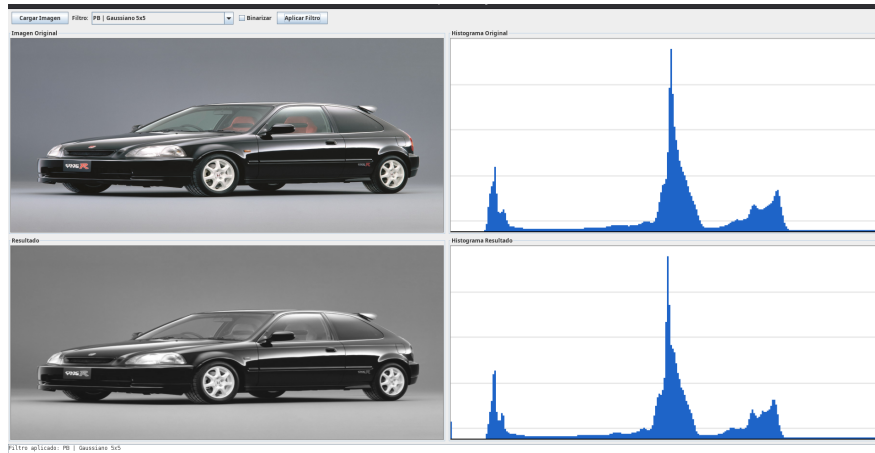


Figura 2: Imagen 2 PB Gaussiano 5×5 .

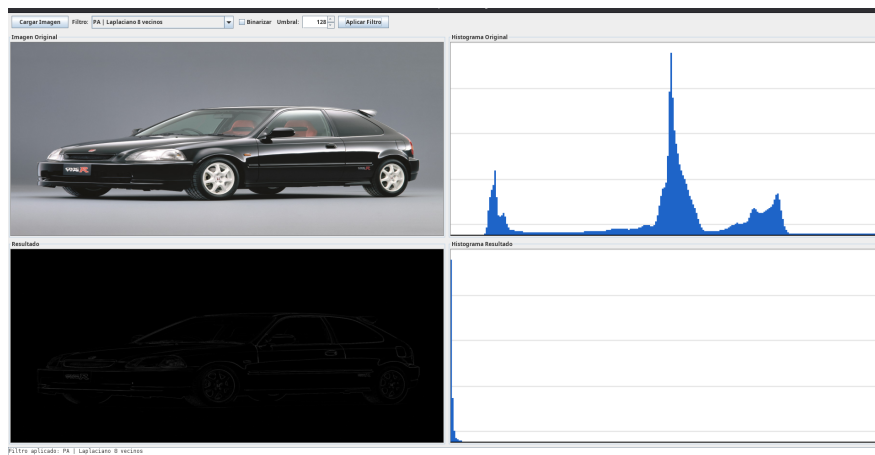


Figura 3: Imagen 3 Laplaciano con 8 vecinos.

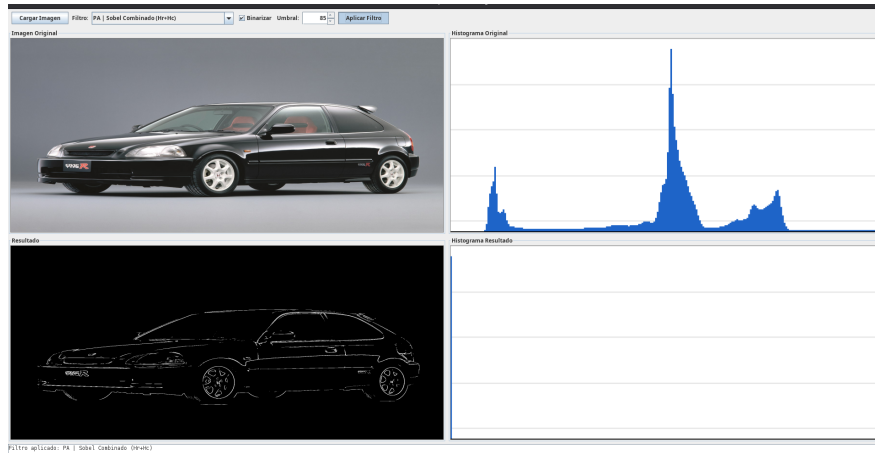


Figura 4: Imagen 4 Sobel combinado .

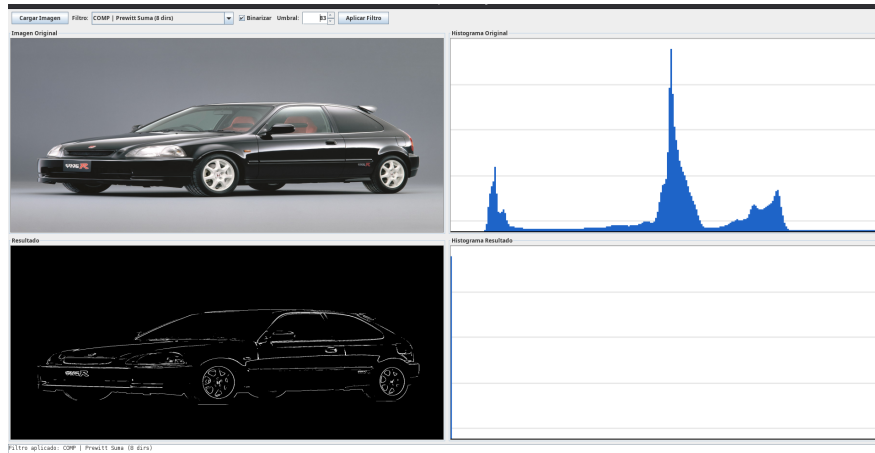


Figura 5: Imagen 5 prewitt suma .

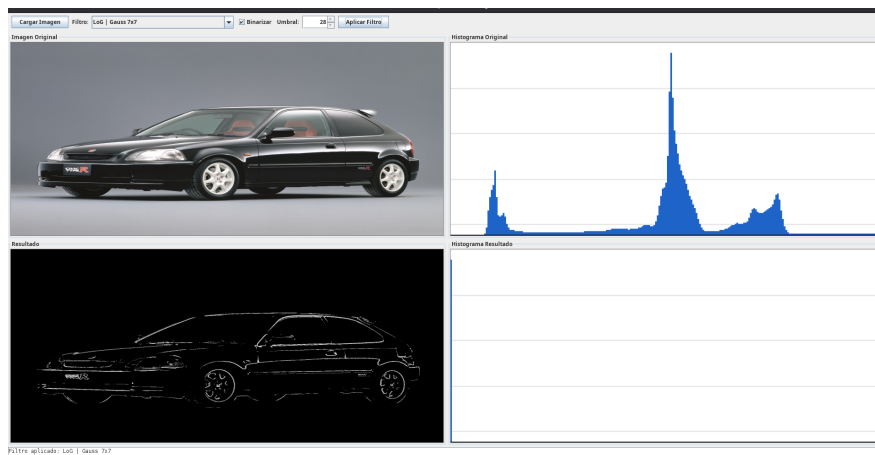


Figura 6: Imagen 6 Gauss 7x7.

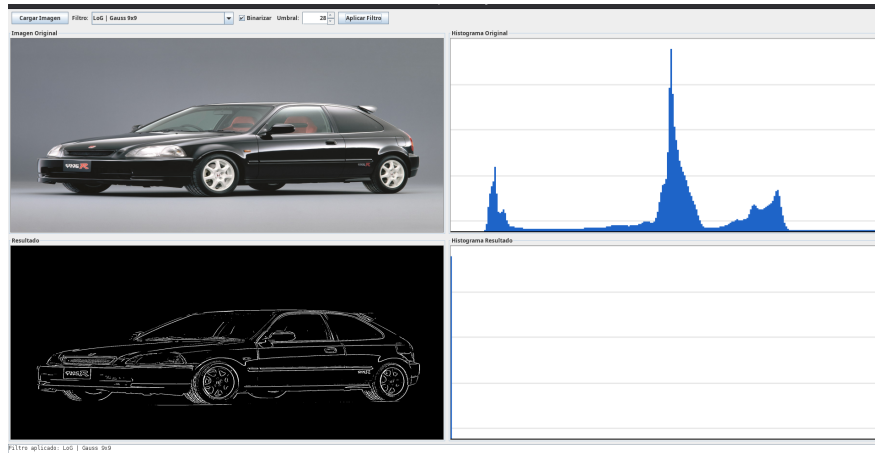


Figura 7: Imagen 7 Gauss 9x9.

3. Conclusión

A través del desarrollo de esta práctica, se comprendió en profundidad el funcionamiento de los filtros espaciales a nivel de matriz y cálculo de píxeles. La implementación manual del motor de convolución demostró que pequeños cambios en los coeficientes del *kernel* pueden generar impactos dramáticos en el análisis visual de la imagen, pasando de un suavizado que difumina imperfecciones a detectores sumamente sensibles al ruido, como los operadores de Roberts o Laplaciano puro.

El uso de operadores combinados como Sobel o Kirsch resulta esencial como paso previo en sistemas de visión artificial y extracción de características, garantizando una segmentación efectiva de los contornos. La estructura MVC elegida facilitó la incorporación escalonada de más de 30 variantes de filtros, comprobando que una buena arquitectura de software es tan vital como el modelo matemático que la sustenta al momento de construir herramientas de análisis digital de imágenes.